

Stock Market Index Direction Prediction Using Limit Order Book Data

MTech (DSAI) Capstone

Joseph Raj Sumanth G.M Abhishek Kumar Kumara Swamy Raghav

June 2026

Abstract

We study short-term *index-direction* prediction from Limit Order Book (LOB) data, combining a reproduction of three published deep LOB models (DeepLOB, MLPLOB, TLOB) on the public FI-2010 benchmark with a first-of-its-kind transfer study on **NSE index futures (NIFTY / BANKNIFTY)**, for which we *engineered our own dataset*. A substantial part of the contribution is **data engineering**: because no clean Indian index-futures LOB dataset existed, we reverse-engineered the undocumented Dhan binary depth feed (two packet formats), built a cloud collection pipeline (EC2 WebSocket collector → daily Parquet → S3) with automated token refresh and expiry-roll handling, and produced a continuous front-month series through a documented preprocessing pipeline that aligns the NSE data to the FI-2010 contract.

On the modelling side we introduce **MambaLOB**, a selective state-space (SSM) architecture with $O(L)$ scaling, and characterise its efficiency–accuracy trade-off against the transformer baseline. We then deepen the data-science contribution in three stages: (i) **microstructure feature engineering** (order-flow imbalance, depth imbalance, micro-price, and the Dhan-unique order-count channel), shown to give a *statistically significant* accuracy gain; (ii) an improved architecture, **ConvMambaLOB**, that fuses a convolutional spatial front-end with the Mamba core; and (iii) a rigour layer of multi-seed confidence intervals, paired significance tests, and probability calibration. The headline finding is: the SSM models recover $\approx 90\%$ of the transformer’s accuracy at $12\text{--}25\times$ fewer parameters - an efficiency result, not an accuracy win - while microstructure features, not architecture, drive the largest significant improvement. We report weighted and macro F_1 side by side throughout and frame transparently what does and does not transfer from the benchmark to the Indian market.

Contents

1	Introduction and Literature Survey	4
1.1	Background and Context	4
1.2	Motivation and Importance of the Study	4
1.3	Problem Definition and Scope	4
1.4	Research Objectives and Questions	4
1.5	Expected Contributions	4
1.6	Review Methodology and Source Selection Criteria	5
1.7	Survey of Existing Approaches	5
1.7.1	Early Solutions and Classical Techniques	5
1.7.2	Modern Approaches and Evolving Trends	5
1.7.3	Emerging Models and Hybrid Frameworks	5
1.8	Comparative Evaluation of Techniques	6
1.9	Tools, Frameworks, and Experimental Protocols	6
1.10	Evaluation Metrics Used in Literature	6
1.11	Limitations of Prior Work	6
1.12	Research Gaps and Open Questions	6
1.13	Innovation and Conceptual Synthesis	7
1.14	Visualizations and Knowledge Summaries	7
1.15	Academic Integrity and Citation Style	7
1.16	Summary of the Chapter	7
1.17	Structure of the Report	7

2	Foundations	7
2.1	Introduction to the Foundations Chapter	7
2.2	Theoretical Underpinnings	7
2.2.1	Market Microstructure	7
2.2.2	The Efficient Market Hypothesis and Its Limits	8
2.3	Conceptual Framework	8
2.3.1	Feature Representation	8
2.3.2	Baseline Models	8
2.3.3	Proposed MambaLOB Model	8
2.3.4	Training and Evaluation	8
2.4	Terminology and Notational Conventions	8
2.4.1	Class Labels	8
2.5	Algorithmic Foundations	9
2.5.1	Data Preprocessing Algorithms	9
2.5.2	DeepLOB / MLPLOB / TLOB	9
2.5.3	Bilinear Normalization and Attention	9
2.5.4	Selective State-Space Modelling	9
2.6	Classification Algorithm	10
2.7	Model Design Principles	10
2.8	Domain-specific Constraints and Assumptions	10
2.9	Ethical, Legal, and Societal Implications	10
2.10	Summary of the Chapter	10
3	Interim Report	10
3.1	Chapter Overview	10
3.2	Dataset Acquisition and Preprocessing	10
3.2.1	Dataset Description	10
3.2.2	Data Quality Checks	11
3.2.3	Exploratory Data Analysis (EDA)	11
3.2.4	Pre-processing Pipelines	11
3.3	Feature Engineering	12
3.3.1	Feature Identification Techniques	12
3.3.2	Dimensionality Reduction	13
3.3.3	Feature Impact Observations	13
3.4	Technical Implementation	14
3.4.1	Tools and Technology Stack	14
3.4.2	Modular Code-base Description	15
3.5	Baseline Model Development	16
3.5.1	Baseline Algorithm Description	16
3.5.2	Training Setup and Parameters	16
3.5.3	Early Performance Metrics — FI-2010 reproduction	16
3.5.4	Output Visualization and Interpretation	16
3.6	Preliminary Analysis and Observations	16
3.6.1	Result Analysis and Discussion	16
3.6.2	Error Profiling and Bias Detection	17
3.6.3	Lessons Learned	17
3.6.4	Improved MambaLOB Architectures (Tier-B)	17
3.6.5	Statistical Significance and Calibration (Tier-C)	19
3.6.6	Hyperparameter Optimization and Class Imbalance (Tier-C)	20
3.7	Originality and Innovation in Approach	21
3.7.1	Creative Framing of the Problem	21
3.7.2	Experimental Innovations	21
3.8	Risk Assessment and Project Management	21
3.8.1	Progress Tracker vs. Original Timeline	21
3.8.2	Bottlenecks and Delays	21
3.8.3	Risk Mitigation Strategies	21
3.9	Experimental Reproducibility	21
3.9.1	Data Splits and Control Measures	21

3.9.2	Random Seed Control	22
3.10	Scalability and Efficiency Considerations	22
3.10.1	Memory, Time, and Resource Profiling	22
3.10.2	Scalability Constraints	22
3.11	Ethical Considerations and Responsible AI Use	23
3.11.1	Generative AI Usage Disclosure	23
3.11.2	Bias and Fairness in Dataset	23
3.12	Feedback Incorporation and Continuous Improvement	23
3.12.1	Peer and Mentor Feedback Summary	23
3.12.2	Reflection and Learning Journey	23
3.13	Chapter Summary and Next Steps	23
A	Detailed Result Tables	24
A.1	NSE Scheme-A transfer matrix (32 runs)	24
A.2	Tier-A microstructure feature ablation (12 runs)	25
A.3	Tier-B architecture comparison (20 runs)	25
A.4	Tier-C multi-seed study (48 runs)	25

1 Introduction and Literature Survey

1.1 Background and Context

Financial markets facilitate capital allocation, risk distribution, and continuous price discovery. Early forecasting relied on statistical time-series models applied to closing prices. Modern electronic exchanges instead operate as continuous double auctions recorded in a *Limit Order Book* (LOB), which updates at microsecond granularity and produces millions of structured events per session. For an index such as **NIFTY 50** - India's premier equity index - the LOB of the index-*futures* contract encapsulates the collective order flow of a broad basket of large-cap stocks, offering a rich microstructural signal for short-horizon direction prediction.

1.2 Motivation and Importance of the Study

Research on deep LOB models is concentrated on developed markets (NASDAQ, Nordic exchanges). Indian index futures are highly liquid yet structurally distinct (tick discreteness, regime shifts, distinct liquidity profiles), and remain almost entirely unstudied with modern LOB architectures. Index direction is hard to predict because the series is non-stationary, exhibits stochastic volatility, and is governed by rapidly evolving supply-demand dynamics. The order book updates irregularly and at high frequency as limit orders are placed, executed, or cancelled, yet most prior deep LOB models train on coarse fixed-grid snapshots (e.g. one-second sampling) that lose intra-interval dynamics. We capture NSE depth at the broker's native push cadence (≈ 2.5 Hz, a ≈ 0.4 s median inter-update gap; Section 3.2), finer-grained than the one-second snapshots common in prior work, while noting transparently that it remains a near-regular snapshot feed rather than tick-by-tick data (a limitation analysed in Section 3.2). The study's principal gap is therefore *geographic*: Indian index futures are highly liquid yet almost entirely unstudied with modern LOB architectures.

1.3 Problem Definition and Scope

We predict the direction of the future mid-price over a short horizon from a window of recent LOB states. The scope comprises: (i) high-frequency LOB inputs (top- k bid/ask prices and volumes); (ii) construction of an event-aligned NSE index-futures dataset; (iii) reproduction of CNN, MLP, and Transformer baselines; (iv) a novel selective-SSM architecture (MambaLOB).

1.4 Research Objectives and Questions

Objectives.

- Reproduce DeepLOB, MLPLOB and TLOB on FI-2010 and validate against published numbers.
- Engineer a reproducible, high-cadence 20-level NSE index-futures LOB dataset (NIFTY, BANKNIFTY).
- Introduce MambaLOB and characterise its accuracy and computational efficiency.
- Compare all architectures on identical data, labels, and protocol; quantify transfer to NSE.

Research Questions.

1. Do our reproductions match published FI-2010 results within tolerance?
2. Is a linear-time selective SSM (Mamba) competitive with a transformer at lower compute?
3. Does LOB directional signal transfer from FI-2010 to NSE index futures, and how much does it degrade?
4. Which labelling scheme (fixed-threshold vs spread-relative) yields an economically meaningful task?

1.5 Expected Contributions

- **Engineered NSE benchmark** - A continuous front-month NIFTY/BANKNIFTY index-futures LOB dataset built from the Dhan depth feed - to our knowledge the first such Indian index-futures LOB dataset prepared for ML, including the full data-engineering pipeline.
- **Reproduction** - Faithful re-implementations of DeepLOB, MLPLOB and TLOB with results benchmarked against the original papers and against independent reproductions (LOBCAST).
- **Novel architecture** - MambaLOB, a selective state-space model for LOB, with a rigorous efficiency comparison (linear vs quadratic scaling in sequence length).
- **Transfer study.** A standardized comparison of four architectures on the NSE benchmark, with honest reporting of degradation relative to FI-2010.

1.6 Review Methodology and Source Selection Criteria

The survey covers classical statistical approaches (ARIMA), Hidden Markov Models, deep LOB models (CNN, LSTM, CNN-LSTM hybrids), the transformer-based TLOB/MLPLOB family, state-space models for sequences (S4/S5/Mamba), and continuous-time microstructure models. Selection criteria: use of high-frequency LOB data, explicit modelling of LOB structure, empirical validation, and architectural novelty. Primary references include the DeepLOB family [1], the TLOB paper [2], FI-2010 (Ntakaris et al. [3]), the LOBCAST benchmark study (Prata et al.) [4], the Mamba paper (Gu & Dao) [5], and continuous-time LOB modelling work.

1.7 Survey of Existing Approaches

1.7.1 Early Solutions and Classical Techniques

Parametric time-series models (AR, MA, ARMA, ARIMA) were the first tools applied to price forecasting. They assume a specific linear data-generating process and perform adequately on smooth, stationary series, but they fail under the non-linear, heavy-tailed, and non-stationary behaviour of high-frequency LOB data, and they ignore book depth entirely, modelling only the price series. Hidden Markov Models and hybrid HMM/ARIMA variants add discrete regime structure (e.g. trending vs. mean-reverting states) and improve over plain ARIMA, but they still operate on derived price/return series rather than the order book, and their performance is sensitive to regime misclassification. These methods form the conceptual baseline against which deep LOB models are judged; none exploit the multi-level bid/ask structure that carries the short-horizon signal.

1.7.2 Modern Approaches and Evolving Trends

The FI-2010 benchmark (Ntakaris et al., 2018) [3]. FI-2010 is the first public benchmark dataset for LOB mid-price forecasting: ten levels of depth for five Nasdaq Nordic (Helsinki) stocks over ten trading days (≈ 4 million events). It standardised the field by providing pre-normalised feature variants (z-score, min-max, decimal precision), a smoothed mid-price percentage-change labelling rule at horizons $k \in \{10, 20, 30, 50, 100\}$, and anchored cross-validation folds. Almost every subsequent deep LOB model — including ours — reports on FI-2010, which is why we use it to validate our reproductions before moving to NSE.

DeepLOB (Zhang, Zohren & Roberts, 2019) [1]. The seminal deep LOB model. It applies a convolutional stack that first fuses price and volume, then the two book sides, then aggregates across levels; an Inception module captures multi-scale temporal patterns; and an LSTM models the resulting sequence before a three-class softmax. On FI-2010 it substantially outperformed earlier shallow methods and generalised to London Stock Exchange stocks. Crucially for this work, it established the interleaved 40-feature $\times$ time-window input representation that all subsequent models (and ours) adopt. Its limitation is the sequential LSTM bottleneck and the absence of attention, which later transformer and state-space models address.

1.7.3 Emerging Models and Hybrid Frameworks

MLPLOB and TLOB (Berti & Kasneci, 2025) [2]. This pair reframes LOB prediction as a stack of normalization-plus-mixing blocks. MLPLOB uses pure MLP mixing with Bilinear Normalization (BiN) and is a deliberately simple but strong baseline. TLOB adds *dual* self-attention — alternating a *temporal* attention (mixing across time steps) with a *feature* attention (mixing across LOB features) — and is the current state of the art on FI-2010, with gains concentrated at longer horizons and in volatile conditions. We reproduce both: MLPLOB as the ablation floor (how much does sequence modelling add?) and TLOB as the SOTA reference, vendored faithfully from the authors' implementation.

State-space models: S4 and Mamba (Gu et al., 2022 [6]; Gu & Dao, 2023 [5]). Structured state-space models (S4) parameterise a linear recurrence with a principled long-range initialisation, achieving sub-quadratic sequence modelling competitive with transformers on long-context tasks. Mamba makes the state-space parameters *input-dependent* (a selection mechanism) and adds a hardware-aware parallel scan, giving genuinely linear-time, $O(L)$ modelling with a parameter count independent of sequence length. SSMs have appeared in LOB research mostly on the generative side; their use as a discriminative *classification backbone* for LOB direction is the specific gap this project targets with MambaLOB.

Benchmark studies and microstructure priors. The LOBCAST benchmark study (Prata et al., 2024) [4] re-implemented fifteen state-of-the-art deep LOB models in a unified framework and found that they overfit FI-2010 and degrade sharply on unseen stocks — a cautionary result that directly motivates our honest transfer framing and our side-by-side reporting of weighted and macro F_1 . Independently of architecture, the market-microstructure literature supplies strong hand-crafted signals: Cont, Kukanov & Stoikov (2014) [7] show that order-flow imbalance is a robust linear predictor of short-horizon price moves, and Stoikov (2018) [8] introduces the micro-price, a volume-weighted fair-value estimator superior to the plain mid. These priors underpin the engineered features evaluated in Section 3.3.

1.8 Comparative Evaluation of Techniques

Table 1: Comparison of LOB modelling approaches considered in this work.

Approach	Strength	Limitation	Role here
ARIMA	Linear forecasting	No non-linear/LOB structure	Conceptual baseline
HMM	Regime modelling	No microstructure depth	Conceptual baseline
DeepLOB (CNN-LSTM)	Depth + temporal	Sequential bottleneck	Reproduced baseline
MLPLOB	Lightweight	No sequence model	Ablation floor
TLOB (Transformer)	SOTA accuracy	$O(L^2)$ attention cost	Reproduced SOTA baseline
MambaLOB (SSM)	$O(L)$, tiny params	Lower accuracy on FI-2010	Proposed model

1.9 Tools, Frameworks, and Experimental Protocols

Python 3.x; PyTorch ($\geq 2.x$) with the CUDA `mamba-ssm` kernel; scikit-learn (metrics, baselines); NumPy/Pandas (LOB processing); AWS EC2 + S3 (collection/storage); Google Colab and Kaggle (GPU training); Matplotlib/Seaborn (figures). Protocol: chronological train/val/test splits; per-feature z-score from training statistics only; sliding windows of length $L = 100$; percentage-mid-change labelling.

1.10 Evaluation Metrics Used in Literature

Table 2: Evaluation metrics for LOB direction prediction.

Metric	Definition	Why it matters
Weighted F1	class-frequency-weighted F_1	matches headline numbers in original papers
Macro F1	unweighted mean F_1	fair under class imbalance; matches independent repro
Accuracy	correct/total	supplementary; misleading under imbalance
MCC	Matthews correlation	balanced single-number summary
Hit-rate / net bps	directional accuracy / cost-adj. return	signal-quality / economic relevance

We report both weighted and macro F_1 throughout, because (as Section 3.6 shows) the original FI-2010 headline numbers are weighted while independent reproductions report macro.

1.11 Limitations of Prior Work

No emerging-market (Indian) application; limited event-driven deep architectures; little use of linear-time state-space models for LOB classification; limited reporting of efficiency/scaling; and inconsistent metric conventions across papers.

1.12 Research Gaps and Open Questions

Can LOB-based models generalise to Indian index futures? Can a linear-time SSM match a transformer at lower compute? How should labels be defined so that the task is economically meaningful (rather than trivially dominated by “no move”)? These motivate our design.

1.13 Innovation and Conceptual Synthesis

This work synthesises classical benchmarking, CNN spatial extraction (DeepLOB), transformer dual-attention (TLOB), and linear-time state-space modelling (Mamba) on a newly engineered Indian benchmark, under one standardized protocol.

1.14 Visualizations and Knowledge Summaries

The report includes comparative tables, architecture descriptions, a data-pipeline schematic, F1-by-horizon charts, confusion matrices, and the memory/latency-vs-sequence-length scaling curve.

1.15 Academic Integrity and Citation Style

Empirical findings are attributed to their source works; conceptual explanations are original synthesis. Generative-AI assistance (Section 3.11.1) was used for drafting and is disclosed.

1.16 Summary of the Chapter

We reviewed classical, deep, transformer, and state-space approaches, identified the Indian-market and efficiency gaps, and positioned our contribution: an engineered NSE LOB benchmark, a reproduction of three baselines, and a novel Mamba-based model evaluated on identical footing.

1.17 Structure of the Report

Chapter 1 (this chapter) introduces the problem and surveys the literature. Chapter 2 establishes the theoretical and architectural foundations. Chapter 3 is the interim report: the engineered dataset, the data-engineering pipeline, preprocessing, baseline development, preliminary results, efficiency analysis, reproducibility, risks, and next steps.

2 Foundations

2.1 Introduction to the Foundations Chapter

This chapter establishes the theoretical underpinnings (market microstructure, the limits of market efficiency), the conceptual data-to-model pipeline, the four model architectures (three reproduced baselines and the proposed MambaLOB), and the notation used throughout.

2.2 Theoretical Underpinnings

2.2.1 Market Microstructure

The continuous double auction maintains a LOB recording all outstanding limit orders at each price level. The *bid* side (buy orders, descending) has best bid P_1^b ; the *ask* side (sell orders, ascending) has best ask P_1^a . The mid-price is the primary, less noisy, forecasting target:

$$p_{\text{mid}}(t) = \frac{P_1^a(t) + P_1^b(t)}{2}. \quad (1)$$

The bid-ask spread $S(t) = P_1^a(t) - P_1^b(t)$ measures liquidity (and, importantly for us, transaction cost), while per-level depth encodes short-term price pressure.

Two microstructure quantities are central to the feature engineering in Section 3.3. *Order-flow imbalance* (OFI; Cont et al. [7]) summarises the net change in bid- versus ask-side liquidity between consecutive states and is one of the most robust documented predictors of short-horizon price moves. The *micro-price* $p_{\text{micro}} = \frac{V_1^b P_1^a + V_1^a P_1^b}{V_1^a + V_1^b}$ (Stoikov [8]) is a volume-weighted mid that leans toward the side with less depth and is often a better instantaneous fair-value estimate than the plain mid. *Depth imbalance*, $\frac{V_1^b - V_1^a}{V_1^b + V_1^a}$, captures the same pressure as a normalised scalar. These priors – rather than raw prices alone – are what the Tier-A features make explicit to the models, and they motivate why engineered features help on the weaker NSE signal.

2.2.2 The Efficient Market Hypothesis and Its Limits

The semi-strong EMH implies no excess return from public information, yet high-frequency studies document short-lived predictability from order-flow imbalance, inventory effects, and latency arbitrage at sub-second to minute horizons. Deep models trained on LOB snapshots exploit precisely these transient inefficiencies; their documented (if small) predictive power motivates this study, while the EMH frames our expectation that signal is weak and easily overwhelmed by costs.

2.3 Conceptual Framework

The pipeline is: *collect* (FI-2010 download; live Dhan capture for NSE) $\rightarrow$ *preprocess* (clean, reorder to a 40-feature LOB vector, z-score on train statistics, label, window) $\rightarrow$ *model* (DeepLOB, MLPLOB, TLOB baselines; MambaLOB proposed) $\rightarrow$ *evaluate* (multi-horizon classification metrics, efficiency, cost-aware backtest). Every model consumes the same tensor $X \in \mathbb{R}^{B \times L \times F}$ ($L = 100$, $F = 40$) and emits logits over three classes.

2.3.1 Feature Representation

Each LOB state is the first ten levels as $[P_i^a, V_i^a, P_i^b, V_i^b]_{i=1}^{10}$ (40 features), the FI-2010 convention. Mid-price and spread are derived from level 1 for labelling and cost accounting.

2.3.2 Baseline Models

DeepLOB (CNN + Inception + LSTM); **MLPLOB** (flatten + MLP + Bilinear Normalization, the ablation floor); **TLOB** (dual temporal/feature self-attention, SOTA). All are reproduced.

2.3.3 Proposed MambaLOB Model

MambaLOB projects the 40-feature input to d_{model} , applies a stack of selective state-space (Mamba) blocks for $O(L)$ temporal mixing, optionally a feature-axis attention, then Bilinear Normalization and a linear classifier:

$$X \xrightarrow{\text{Linear}} \text{Mamba} \times n \xrightarrow{(\text{opt. feature attn})} \text{BiN} \xrightarrow{\text{FC}} \text{logits} \in \mathbb{R}^3.$$

On GPU it uses the CUDA `mamba-ssm` kernel; a pure-PyTorch fallback exists for CPU/debug. Phase-2 variants (a convolutional front-end and a bidirectional scan) build on this core and are detailed with the experiments (Section 3.6, Figure 12).

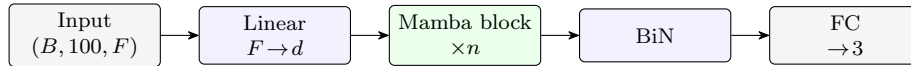


Figure 1: Proposed MambaLOB. The LOB window is projected to d_{model} , passed through n selective state-space (Mamba) blocks for $O(L)$ temporal mixing (block internals in Figure 3), then Bilinear Normalization and a 3-class head.

2.3.4 Training and Evaluation

Supervised three-class classification; class-weighted cross-entropy; AdamW with cosine decay; early stopping on validation macro/weighted- F_1 ; metrics as in Table 2.

2.4 Terminology and Notational Conventions

2.4.1 Class Labels

We use the encoding $\{0 = \text{Down}, 1 = \text{Stationary}, 2 = \text{Up}\}$. Labels are assigned by the smoothed-mid percentage change over the horizon (Section 2.6).

Table 3: Notation.

Symbol	Meaning
$X \in \mathbb{R}^{B \times L \times F}$	input window batch ($L = 100, F = 40$)
L, F, B	sequence length, features, batch size
$d_{\text{model}}, d_{\text{state}}, d_{\text{conv}}$	Mamba projection / state / conv width
k (or H)	prediction horizon in LOB events
p_{mid}, S	mid-price, bid–ask spread
θ, α	label thresholds (spread-relative / fixed)

2.5 Algorithmic Foundations

2.5.1 Data Preprocessing Algorithms

Z-score normalization per feature, using *training* statistics only: $Z = (X - \mu)/\sigma$. **Sliding-window** segmentation produces $X \in \mathbb{R}^{L \times F}$ with $L = 100$. **Labelling**: with $m^-(t)$ and $m^+(t)$ the mean mid over the k events behind/ahead, $r(t) = (m^+ - m^-)/m^-$; then Up if $r > \tau$, Down if $r < -\tau$, else Stationary, where τ is the threshold (Scheme A: fixed α ; Scheme B: spread-relative θ).

2.5.2 DeepLOB / MLPLOB / TLOB

DeepLOB stacks 1×2 convolutions (fuse price/volume, then sides), a 1×10 convolution (fuse levels), an Inception module, and an LSTM. MLPLOB flattens and applies MLP blocks with Bilinear Normalization. TLOB alternates temporal and feature self-attention with a progressive MLP head. Figure 2 shows the three reproduced baselines as block pipelines.

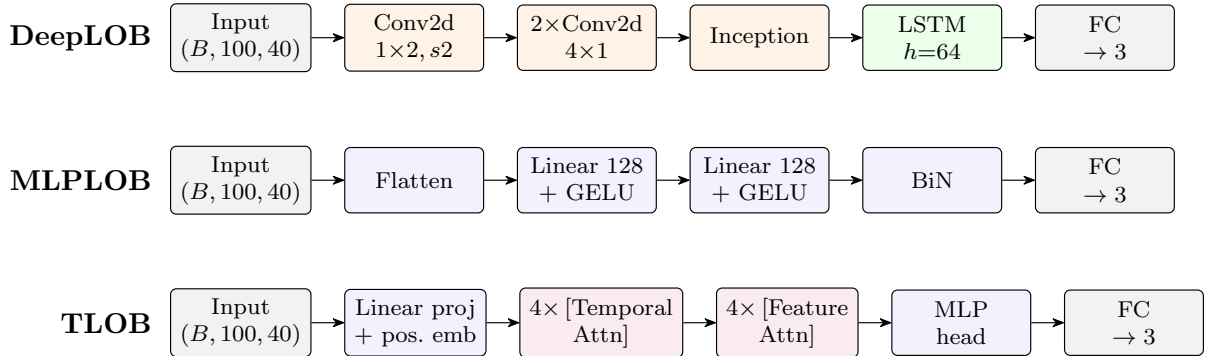


Figure 2: Reproduced baseline architectures. DeepLOB (CNN + Inception + LSTM), MLPLOB (flatten + MLP with Bilinear Normalization, the ablation floor), and TLOB (alternating temporal and feature self-attention, the transformer SOTA). All share the $(B, 100, 40) \rightarrow 3$ contract.

2.5.3 Bilinear Normalization and Attention

Bilinear Normalization (BiN) normalises across both the temporal and feature axes and learns to weight the two, stabilising LOB training. Multi-head self-attention provides cross-position mixing in TLOB; its cost is quadratic in L .

2.5.4 Selective State-Space Modelling

A Mamba block realises a discretised, input-dependent linear recurrence $h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$, $y_t = C_t h_t$, where the selection mechanism makes $\bar{A}, \bar{B}, C$ depend on the input. This yields $O(L)$ time and memory and a parameter count independent of L — the basis of our efficiency claim. Figure 3 details the block internals.

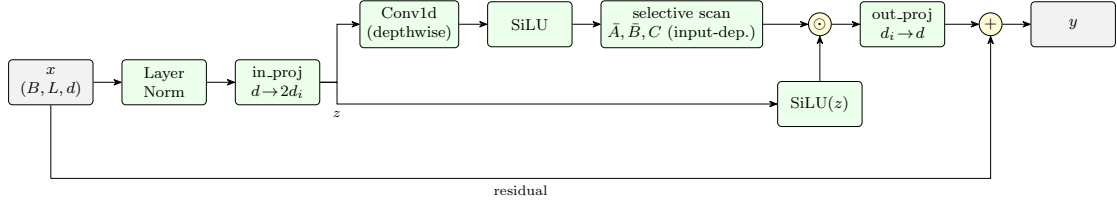


Figure 3: Selective state-space (Mamba) block. The input is normalised and projected to two streams; one passes a depthwise causal convolution and SiLU into the selective scan (with input-dependent $\bar{A}, \bar{B}, C$), the other forms the SiLU gate. The gated output is projected back and added to the residual. The scan is the only sequential step and runs in $O(L)$.

2.6 Classification Algorithm

The head produces $\hat{y} = \text{Softmax}(Wx + b)$ over the three classes. Optimization uses class-weighted cross-entropy and AdamW; evaluation uses weighted/macro F_1 , accuracy, and MCC.

2.7 Model Design Principles

Modularity (separate ingest / dataset / model / train / eval modules), reusability (shared loaders, normalization, windowing across FI-2010 and NSE), adaptability (one data contract for all models), and scalability (lazy windowing keeps memory $O(T \cdot F)$ rather than materialising all windows).

2.8 Domain-specific Constraints and Assumptions

Tick discreteness (NIFTY futures move in ≥ 0.05 increments) quantizes mid-price changes and informs threshold choice; the bid–ask spread sets a hard floor on tradeable signal; pre-open and the opening/closing auction windows are excluded to isolate continuous trading; expiry days require contract rolling.

2.9 Ethical, Legal, and Societal Implications

The work uses public benchmark data and self-collected public market data (no personal data). Predictions are framed as *signal-quality* research, not trading advice.

2.10 Summary of the Chapter

We established microstructure foundations, the shared data-to-model pipeline, the four architectures, and the notation. The next chapter reports the engineered dataset and the empirical results.

3 Interim Report

3.1 Chapter Overview

This chapter documents the work completed to date: (i) the **data-engineering** effort to build an NSE index-futures LOB dataset; (ii) the preprocessing pipeline aligning it to FI-2010; (iii) reproduction of the three baselines and the proposed MambaLOB; (iv) preliminary results on FI-2010 and the NSE transfer; (v) an efficiency analysis; and (vi) reproducibility, risks, and next steps.

3.2 Dataset Acquisition and Preprocessing

3.2.1 Dataset Description

FI-2010 (reproduction). The public NoAuction z-score variant: 5 stocks, 10 days, 10 levels, pre-normalized, with standard cross-folds (we use CF_7: 7 train + 3 test days).

NSE index futures (engineered). We built a live dataset for the NIFTY and BANKNIFTY near-month futures (the most liquid index-derivative contracts on the National Stock Exchange of India) using the Dhan twenty-level depth feed, stored as daily Parquet on Amazon S3. Because the stored feed is a ≈ 2.5 Hz snapshot stream (Section 3.2), an “event” in this work is one recorded top-of-book state, i.e. a

stored depth snapshot, rather than a single exchange message. Key facts: 29 valid trading days (12 May – 18 June 2026), spanning the May→June futures expiry roll; $\approx 57,000$ events/instrument/day (≈ 1.1 M per instrument, exceeding FI-2010’s ≈ 400 k scale); each record carries 20 bid and 20 ask levels with price, quantity, *and* order count. Taking the first 10 of the 20 levels and reordering to the interleaved $[P_i^a, V_i^a, P_i^b, V_i^b]$ layout yields a 40-feature vector identical to FI-2010, so all models transfer to NSE with no architectural change — the design choice that makes a direct benchmark-to-market comparison possible.

3.2.2 Data Quality Checks

We evaluated two Indian data sources before committing. Table 4 summarises the comparison that led us to choose Dhan. Dhan wins on depth (20 vs 5 levels, a direct 40-feature match to FI-2010), a $\sim 2.5\times$

Table 4: Dhan vs Kite vs FI-2010 (15-min simultaneous capture).

Property	Dhan (20-depth)	Kite Connect	FI-2010
LOB levels	20	5	10
Features (10-level)	40 (direct match)	20 (needs change)	40
Update mechanism	push, ~ 0.4 s cadence	periodic ~ 1 s	event-driven
Median inter-tick gap	0.40 s	1.0 s	< 0.1 s
Update rate	~ 2.5 Hz	~ 1 Hz	per-event
L1–L10 completeness	100%	n/a	100%
Order-count field	yes	no	no
Cost	low	higher	free

higher update rate, full L1–L10 completeness, the order-count field, and cost. Beyond source selection, per-day cleaning drops: rows with crossed quotes ($P_1^b \geq P_1^a$); rows with any missing level-1–10 field; and tick-level glitches (mid-price jumps $> 0.5\%$ relative to the previous valid mid). Empty/partial collection days (the broken expiry-roll files of 27–29 May) are excluded; the partial 11 May session is dropped.

3.2.3 Exploratory Data Analysis (EDA)

We characterise the engineered NSE data on the training days (NIFTY front-month). The collector records one row per server push, so the stored cadence reflects the Dhan feed itself: ≈ 154 NIFTY events/min, i.e. a **regular ≈ 0.4 s (≈ 2.5 Hz) update**. The inter-tick gap distribution (Figure 5) is tightly concentrated in 0.2–0.7 s (median 0.40 s, p_{99} 0.60 s, coefficient of variation 0.34) — the twenty-depth feed delivers near-periodic snapshots rather than the heavy-tailed per-event stream of FI-2010. This is adequate for short-horizon prediction but is a genuine difference from the benchmark, and one reason the absolute NSE accuracy is lower (Section 3.6).

The relative bid–ask spread (Figure 5, right) has a median of ≈ 3.3 bps; because the horizon- k mid-price move is often smaller than the spread, the labelling threshold has a decisive effect on the task (Section 3.6, Scheme B). The label distribution (Figure 4) shifts sharply with horizon: under the fixed threshold the Stationary class shrinks from $\approx 19\%$ at $k=10$ to $\approx 4\%$ at $k=100$, so the long-horizon task is effectively binary (Down/Up) — which is why macro and weighted F_1 converge at long horizons. A representative intraday mid-price trace (Figure 6) shows the trending, non-stationary character of a single session.

3.2.4 Pre-processing Pipelines

The pipeline that aligns Dhan to the FI-2010 contract (implemented in `modeling/nse_dataset.py`):

1. **Front-month stitching:** match each daily file by root (NIFTY/BANKNIFTY), concatenating the May and June contracts into one continuous front-month series.
2. **Feature reorder:** map Dhan’s column ranges to the interleaved FI-2010 layout $[P_i^a, V_i^a, P_i^b, V_i^b] \times 10$.
3. **Cleaning:** crossed-quote, missing-level, and tick-spike filters (above).
4. **Per-day segmentation:** labels and windows are computed *within* a trading day, so the overnight gap and the expiry roll are segment boundaries — no spurious cross-day returns and no window straddles two contracts.

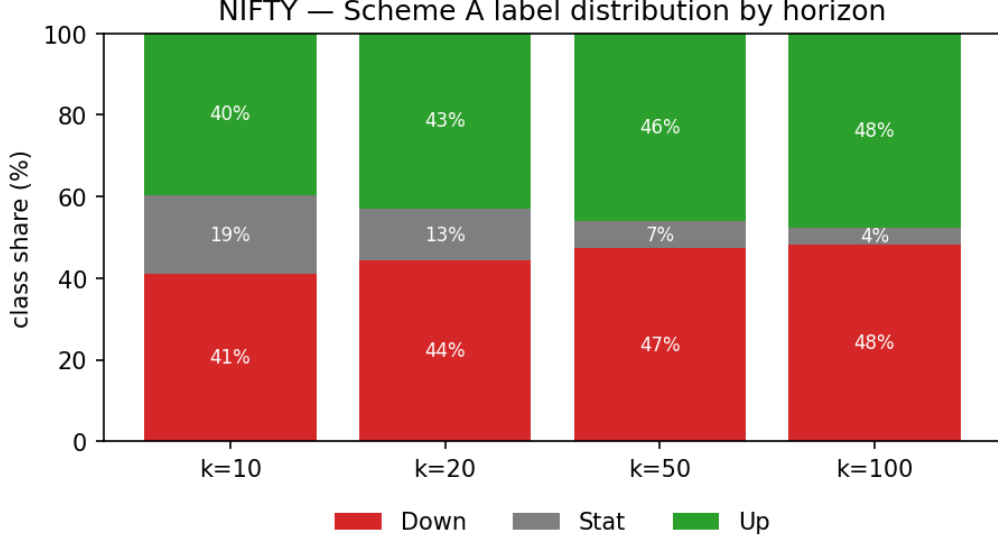


Figure 4: NSE (NIFTY) Scheme-A label distribution by horizon. The Stationary class shrinks from $\approx 19\%$ at $k=10$ to $\approx 4\%$ at $k=100$ as more mid-price moves exceed the fixed threshold.

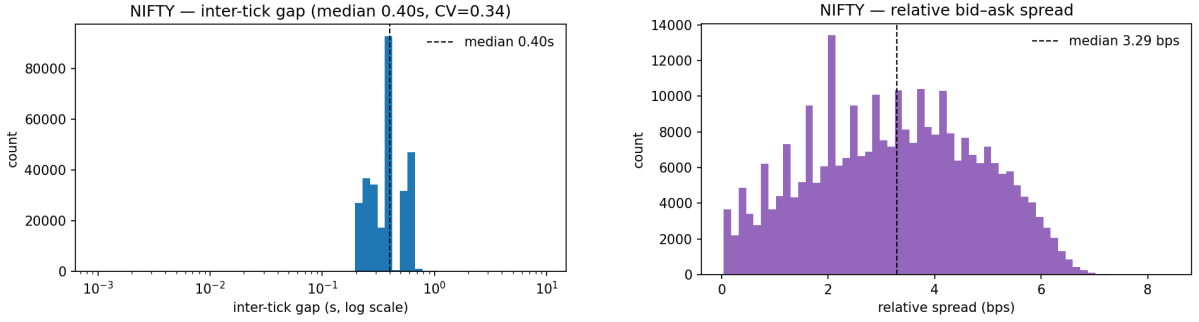


Figure 5: Left: inter-tick gap distribution (log axis) — a regular ≈ 0.4 s cadence (CV 0.34), not heavy-tailed. Right: relative bid-ask spread, median ≈ 3.3 bps.

5. **Z-score** using training statistics only.
6. **Labelling** (Scheme A fixed- α or Scheme B spread-relative).
7. **Lazy sliding windows**: the (T, F) matrix is stored once and windows are sliced on demand, keeping memory $O(T \cdot F)$; the eager (N, L, F) expansion would exhaust memory on the full series. Default split (chronological): train 12 May – 4 June (15 days), validation = last 10% of training windows, test 5/8/9/10/11/12 June (6 most-recent out-of-sample days). Figure 7 shows the end-to-end data-engineering flow from the live broker feed to model-ready tensors.

3.3 Feature Engineering

3.3.1 Feature Identification Techniques

The base model input is the raw 40-dimensional LOB vector (10 levels) per event. On top of this we engineer *microstructure* channels (`modeling/features.py`), motivated by the weak raw-LOB signal on NSE:

- **Order-Flow Imbalance (OFI)** (Cont et al.) at level 1 and summed over levels 1–5, computed per trading day so flow never crosses an overnight/expiry boundary;
- **depth imbalance** (level 1 and aggregate over 10 levels), **micro-price** (Stoikov, volume-weighted mid), and **relative spread**;
- **per-level order counts** — a channel *unique to the Dhan feed*, absent from FI-2010 and Kite.

These define named feature sets used for ablation: **base40** (raw, 40), **micro** (+6 scalars, 46), **orders60** (+order counts, 60), **depth80** (20 levels, 80), and **all** (66).

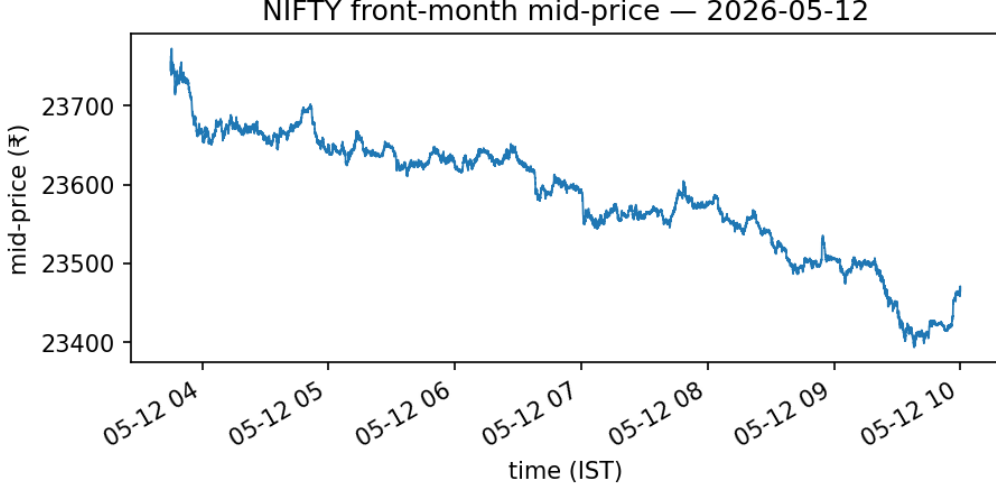


Figure 6: Representative intraday front-month mid-price (NIFTY, 12 May 2026), illustrating the trending, non-stationary character of a single trading session.

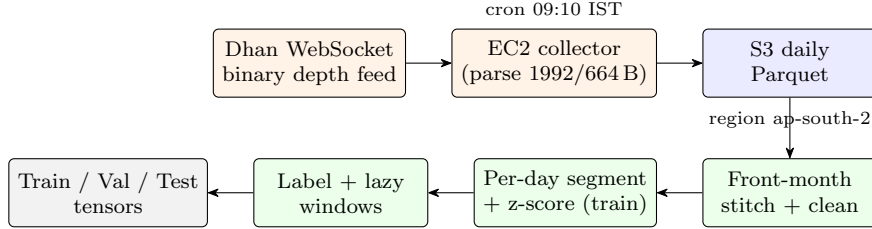


Figure 7: Engineered NSE data pipeline. The reverse-engineered Dhan binary feed (two frame formats, 1992 B full / 664 B compact) is parsed on an EC2 collector and written as daily Parquet to S3; the modelling layer then stitches the front-month series across the expiry roll, cleans, segments per trading day, z-scores using training statistics only, labels, and emits lazily-windowed tensors aligned to the FI-2010 layout.

3.3.2 Dimensionality Reduction

None applied; the input is intentionally the raw/engineered LOB tensor. We instead study which *representation* the signal needs via the depth choice and the feature-set ablation below.

3.3.3 Feature Impact Observations

Table 5 shows the ablation for the proposed MambaLOB on NIFTY (improvement in test weighted- F_1 over **base40**). Findings: (i) **microstructure features help** — **micro** and **all** beat raw LOB, with

Table 5: Feature ablation — MambaLOB (NIFTY), Δ weighted- F_1 vs **base40**.

Feature set	Δ (k=10)	Δ (k=100)	Verdict
micro	+0.028	+0.040	helps
orders60	−0.069	−0.007	hurts alone
depth80	−0.003	+0.002	$\approx$ neutral
all	+0.024	+0.068	best

the gain growing with horizon (at k=100 the **all** set lifts weighted- F_1 from 0.569 to 0.637); the effect also holds for TLOB (+0.007). (ii) **Raw order counts help only in combination** with the micro scalars (**orders60** alone hurts, but **all** is best) — a feature-interaction result that exploits the Dhan-unique channel. (iii) **Book depth beyond level 10 is uninformative** (**depth80** neutral) — the NSE index-futures signal lives at the top of book. (iv) The features add negligible parameters (68k $\rightarrow$ 70k),

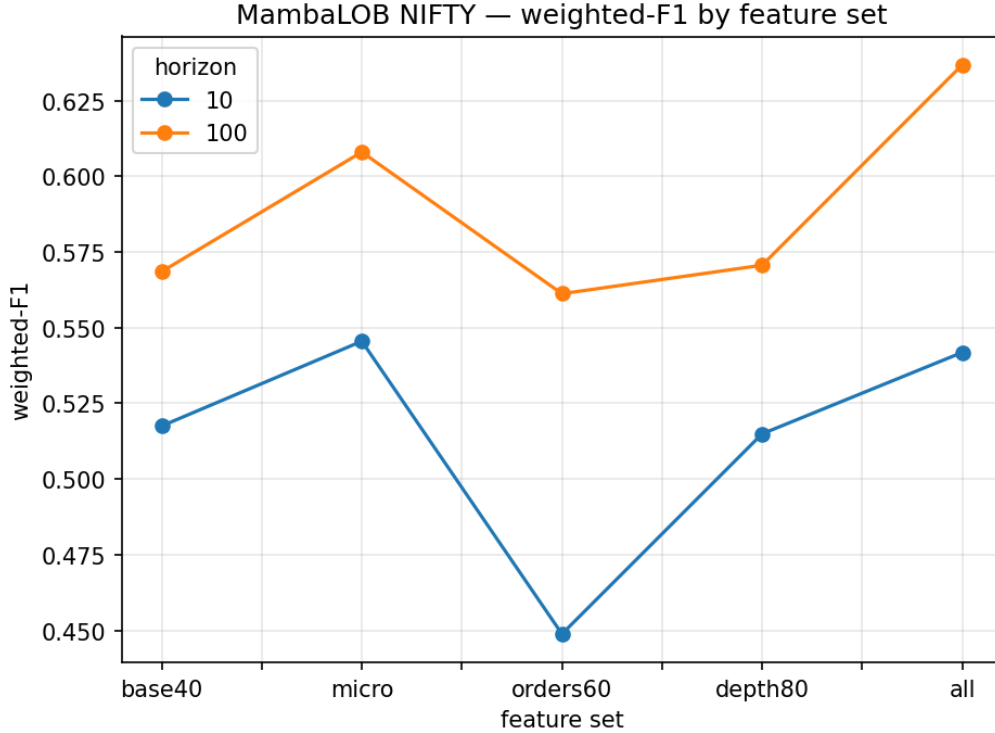


Figure 8: MambaLOB (NIFTY) weighted- F_1 by feature set and horizon. The engineered `micro` and `all` sets sit above raw `base40`, with the gain widening at the longer horizon; deep levels (`depth80`) and order counts alone (`orders60`) do not help in isolation.

so MambaLOB stays the efficient model. The separate labelling-threshold study (Section 3.6) remains the key *economic* finding. The multi-seed study (Section 3.6, Table 10) confirms this feature-engineering gain is statistically significant ($p=0.001$).

3.4 Technical Implementation

3.4.1 Tools and Technology Stack

Python; PyTorch with `mamba-ssm/causal-conv1d` (CUDA); scikit-learn; NumPy/Pandas; Dhan and Kite WebSocket APIs; AWS EC2 (collection) and S3 (storage, region `ap-south-2`); Google Colab and Kaggle (GPU training); `boto3/s3fs`; Matplotlib/Seaborn; Git/GitHub for version control.

Because no clean Indian index-futures LOB dataset existed, the dataset was *engineered end to end*. This was the most substantial part of the project and is described in three parts below: decoding the broker’s binary protocol, building a robust cloud collection service, and operating it reliably across the contract-expiry roll.

Reverse-engineering the Dhan binary depth feed. The Dhan twenty-level depth WebSocket delivers market data as an *undocumented binary frame* rather than JSON. We decoded its structure offline by capturing raw frames and correlating fields against the live order book. The feed uses two frame formats (Table 6): a 1992-byte *full* packet (a 40-byte header followed by twenty 16-byte bid records and twenty 16-byte ask records) and a 664-byte *compact* packet carrying two 332-byte sub-packets. Each level record packs an order-count, an IEEE-754 double price, and a quantity. Decoding surfaced several non-obvious issues that had to be fixed for the data to be usable: the bid price sits at byte offset +4 within the record (not +0, as a naive read assumes); empty levels emit *denormalised* near-zero floats that must be rejected rather than parsed as ≈ 0 prices; and quantity/order-count fields can exceed the signed 32-bit range, so the Parquet schema uses `int64` (a `uint32/int32` choice silently overflows). The parser is greedy — on an unrecognised frame length it advances one byte and re-tries — which keeps the stream alive through occasional malformed packets. Decoding the binary protocol was the single hardest engineering task in the project and is what makes the twenty-level Indian dataset

possible at all.

Table 6: Decoded Dhan twenty-depth binary frame layout.

Element	Size	Contents
Full frame	1992 B	40 B header + 20 bid records + 20 ask records
Compact frame	664 B	two 332 B sub-packets
Level record	16 B	order-count, price (f64, offset +4), quantity
Schema quirk	—	qty / orders stored as <code>int64</code> (<code>uint32</code> overflows)
Empty level	—	denormalised near-zero float $\Rightarrow$ rejected

Cloud collection architecture. A lightweight EC2 instance (Ubuntu, `ap-south-2/Hyderabad`) runs an `asyncio` WebSocket collector. It subscribes to the NIFTY and BANKNIFTY near-month futures on the `NSE.FNO` segment, records *one row per server push* (no polling timer), and buffers rows to a daily `zstd`-compressed Parquet file, flushing in batches to bound memory. Timestamps are stored timezone-aware (parsed with `format='ISO8601'`, required for pandas 2.x). The collector is gated to market hours (09:15–15:30 IST) and tolerates the broker’s keepalive-timeout disconnect at the close as a normal stop. A cron schedule starts the collector at 09:10 IST (03:40 UTC) and syncs the day’s Parquet to S3 at 15:40 IST (10:10 UTC); only Parquet is synced, keeping the bucket clean. The result is $\approx 57,000$ events/instrument/day at ≈ 2.5 Hz.

Daily operations, robustness, and the expiry roll. Operating a live feed surfaced realities that a static dataset hides. Broker access tokens expire at midnight IST, so a refresh script regenerates the Dhan and Kite tokens each morning before the open. The WebSocket reconnects with a backoff on transient drops. Two production bugs were diagnosed and fixed: the futures segment code is `NSE.FNO` (not `NSE.FO`), and an instrument was briefly mislabelled after a broker reassigned a security token following a 1:1 bonus issue. The most consequential operational event was the **May→June contract expiry roll**: the May future expired and stopped trading, collection rolled to the June contract, and the broken collection files around the roll (27–29 May) are zero-byte and excluded. The loader stitches the May and June contracts into one continuous front-month series *by root symbol*, treating the roll (and every overnight gap) as a segment boundary so that no window or label ever straddles two contracts.

Training environments. FI-2010 reproduction and the Mamba/NSE experiments run on Google Colab and Kaggle GPUs. Secrets (GitHub PAT, AWS keys, broker tokens) are read host-agnostically from Kaggle/Colab secret stores or the environment, so the same notebooks run unchanged on either platform. Every experiment uploads its results, metrics JSON, and checkpoints to S3 *as each run completes*, making long matrices timeout-safe and resumable across sessions — a session interruption costs at most the single run in flight.

3.4.2 Modular Code-base Description

The repository (`nse-lob-capstone`) separates concerns into clearly bounded modules:

- **data_acquisition/** — the data-engineering layer: `collect_dhan.py` (binary parser + collector), `collect_kiteconnect.py`, token-refresh, S3-sync, and EC2 operational notes.
- **modeling/** — datasets (`fi2010_dataset.py`, `nse_dataset.py`), models (`models.py`: DeepLOB/MLPLOB/Mamba and the Phase-2 ConvMambaLOB / bidirectional variants; `tlob.py`: vendored TLOB), engineered features (`features.py`), losses (`losses.py`: focal), training/eval (`train.py`), the unified experiment runner and dual-GPU shard launcher (`runner.py`, `run_shard.py`), HPO worker (`run_hpo.py`), statistics (`stats.py`: seeding + bootstrap CIs), the cost-aware backtest (`backtest.py`), and a host-agnostic notebook/secret/S3 helper (`nbenv.py`).
- **notebooks/** — one notebook per experiment: FI-2010 reproduction, TLOB, Mamba+efficiency, the NSE Scheme-A matrix and extras, the Tier-A feature ablation (dual-GPU), Tier-B architectures, the Tier-C multi-seed rigour study, the Tier-C HPO + class-imbalance study, and the standalone EDA notebook. Executed copies and HTML exports live under `notebooks/from_kaggle/`.
- **literature/** (paper notes and survey) and **docs/** (plans, work log, and this report).

All models consume one (B, L, F) data contract, so a single loader and trainer serve both FI-2010 and NSE, and every model is swappable through a name-keyed registry (`build_model`).

3.5 Baseline Model Development

3.5.1 Baseline Algorithm Description

All baselines implement the shared $(B, 100, 40) \rightarrow [B, 3]$ contract (Section 2.5, Figure 2) so they are directly comparable on identical inputs, labels, and protocol. **DeepLOB** (Zhang et al.) stacks 1×2 convolutions that fuse price/volume then the two book sides, a level-fusing convolution, an Inception module for multi-scale temporal patterns, and an LSTM; we re-implemented it from the paper. **MLPLOB** (Berti & Kasneci) flattens the window and applies MLP blocks with Bilinear Normalization, serving as the ablation floor that isolates how much temporal structure matters. **TLOB** (the transformer SOTA) alternates temporal and feature self-attention with a progressive MLP head; to guarantee fidelity we *vendored* the official implementation into `modeling/tlob.py`, making it self-contained (no external `einops`/constants) and device-agnostic rather than re-deriving it. This faithful-reproduction-first stance is what lets us trust the FI-2010 numbers as a validation of the pipeline before introducing the proposed models.

3.5.2 Training Setup and Parameters

Max 50 epochs (FI-2010) / 20 epochs (NSE), early stopping on validation F_1 ; AdamW; class-weighted cross-entropy; batch 128; per-model learning rates following the papers (DeepLOB/MLPLOB 10^{-3} , TLOB 10^{-4} , MambaLOB 3×10^{-4}).

3.5.3 Early Performance Metrics — FI-2010 reproduction

Table 7: FI-2010 (CF_7, $L = 100$) — weighted / macro F_1 (%).

Model	$k=10$	$k=20$	$k=50$	$k=100$	Notes
DeepLOB	82.5 / 71.9	—	—	—	$\approx$ paper 83.4 (weighted)
TLOB	88.0 / 81.1	77.3 / 70.2	88.4 / 87.5	92.5 / 92.5	SOTA-range; paper ~ 92.8 @ seq 128
MambaLOB (M1)	79.2 / 67.4	71.1 / 61.4	72.8 / 70.0	69.3 / 68.7	lower accuracy, far cheaper

Acceptance. DeepLOB’s weighted F_1 (82.5%) matches the paper’s headline ($\sim 83.4\%$); TLOB is in the SOTA range, reaching 92.5% at $k=100$ (the paper’s ~ 92.8 was obtained at sequence length 128; we use 100). These confirm the reproduction is correct.

3.5.4 Output Visualization and Interpretation

We report confusion matrices per (model, horizon), F_1 -by-horizon line charts, and the efficiency scaling curve (Section 3.10). Figure 9 shows the FI-2010 macro- F_1 trend across horizons; Figure 10 the DeepLOB confusion matrices at the shortest and longest horizons.

3.6 Preliminary Analysis and Observations

3.6.1 Result Analysis and Discussion

FI-2010. On accuracy, TLOB $>$ DeepLOB $>$ MambaLOB. **NSE transfer (Scheme A, complete — 32 runs over 4 models $\times$ 2 symbols $\times$ 4 horizons).** Table 8 summarises weighted F_1 . All models beat the naive baselines (random ≈ 0.33 ; lift over the best baseline is positive everywhere), so there *is* learnable signal on NSE — but the absolute numbers are well below FI-2010, the expected degradation on new data (consistent with the LOBCAST benchmark study).

Table 8: NSE Scheme A — test weighted F_1 (range over $k \in \{10, 20, 50, 100\}$). Lift = weighted F_1 minus the best naive baseline.

Model	NIFTY (wF1)	BANKNIFTY (wF1)	Lift over baseline
DeepLOB	0.44–0.52	0.46–0.50	0.03–0.12
MLPLOB	0.46–0.64	0.35–0.47	0.01–0.19
MambaLOB	0.52–0.59	0.45–0.48	0.02–0.17
TLOB	0.60–0.71	0.64–0.72	0.25–0.30

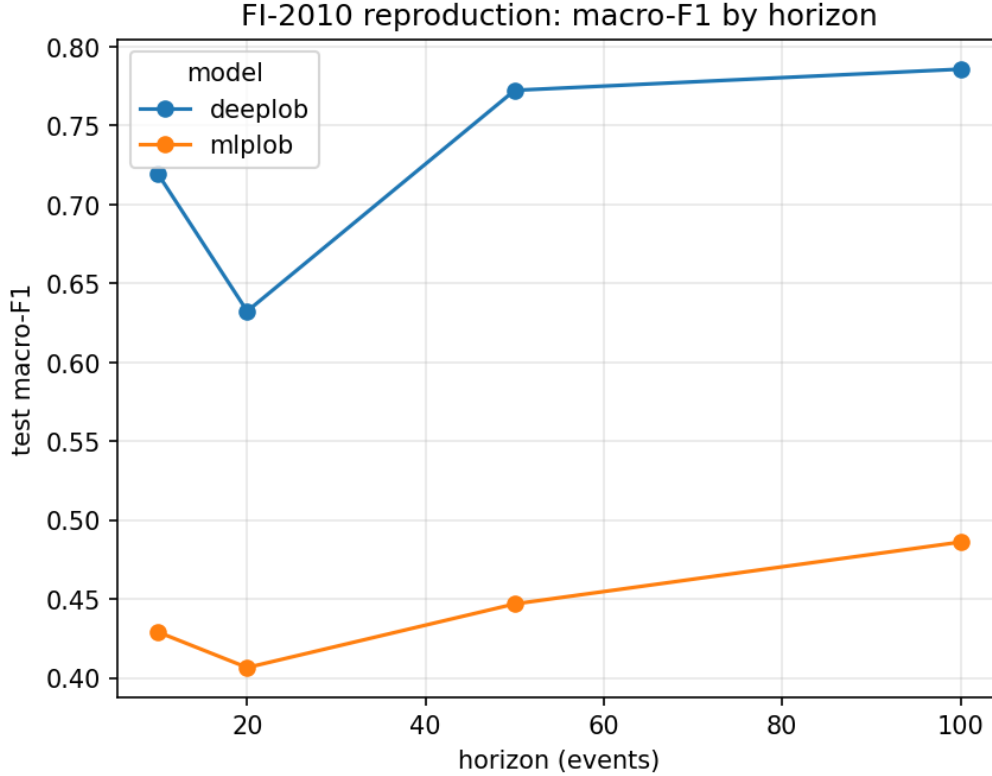


Figure 9: FI-2010 reproduction: macro- F_1 by prediction horizon for the reproduced baselines. The ordering and absolute range match the published DeepLOB/TLOB results and the independent LOBCAST reproduction.

TLOB leads across both instruments and all horizons (lift ≈ 0.25 – 0.30), mirroring its FI-2010 ranking. The key transfer finding concerns the proposed model: on NIFTY, MambaLOB (68k parameters) outperforms DeepLOB (191k) and matches MLPLOB, second only to TLOB, whereas on FI-2010 MambaLOB trailed DeepLOB. In other words, on the target Indian market the selective-SSM transfers *better* than on the benchmark, delivering competitive accuracy at roughly 1/27 of TLOB’s parameters. On BANKNIFTY MambaLOB is on par with DeepLOB (both ≈ 0.46 – 0.48), below TLOB. Macro F_1 (NIFTY) follows the same order: DeepLOB 0.39–0.42, MLPLOB 0.44–0.47, MambaLOB 0.44–0.50, TLOB 0.53–0.58.

3.6.2 Error Profiling and Bias Detection

The dominant effect is class balance. Under the fixed threshold the Stationary class is small at long horizons (near-binary), whereas under the spread-relative scheme it dominates. Confusion matrices show errors concentrated on the minority directional classes, which is why macro F_1 trails weighted F_1 .

3.6.3 Lessons Learned

(1) **Metric convention matters:** the FI-2010 headline (~ 83.4) is a *weighted* F_1 ; our weighted F_1 matches it, while our macro F_1 (≈ 72) matches *independent* reproductions (LOBCAST). We therefore report both. (2) **$F_1 \neq \text{profit}$:** see Section 3.6 / 3.10. (3) **Mamba is primarily an efficiency win:** it trails on FI-2010 accuracy, yet on the NSE transfer it is competitive — beating the CNN baseline (DeepLOB) on NIFTY — at a fraction of the parameters and compute.

3.6.4 Improved MambaLOB Architectures (Tier-B)

We tested two architectural improvements to the vanilla SSM stack — *bidirectional* Mamba (a forward and a time-reversed scan, averaged) and *ConvMambaLOB* (DeepLOB’s Conv+Inception spatial front-end feeding a Mamba temporal core, replacing DeepLOB’s sequential LSTM) — plus their combination,

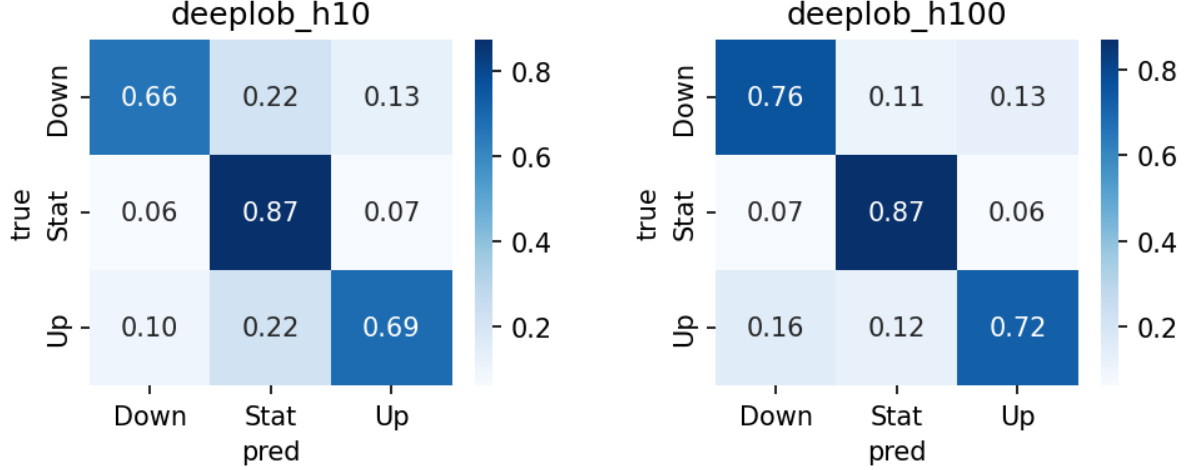


Figure 10: DeepLOB confusion matrices on FI-2010 at $k=10$ (left) and $k=100$ (right). At the longer horizon the Stationary class shrinks and predictions concentrate on the directional classes.

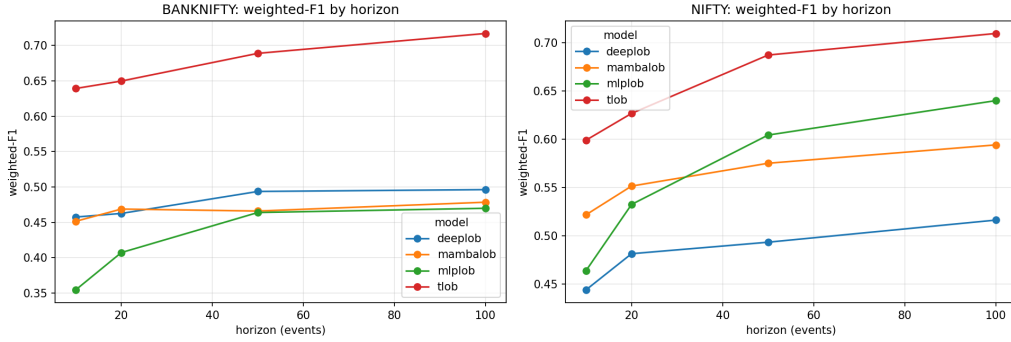


Figure 11: NSE transfer (Scheme A): test weighted- F_1 by horizon for each model. TLOB leads throughout; MambaLOB transfers competitively, matching or beating the CNN baseline on NIFTY at a fraction of the parameters.

all trained on the best (a11) feature set against the TLOB transformer. Figure 12 contrasts the four variants; Table 9 reports the mean over both instruments and $k \in \{10, 100\}$.

Table 9: Tier-B architectures (NIFTY+BANKNIFTY, mean over $k \in \{10, 100\}$, a11 features, single seed).

Model	Params	wF1	mF1	% of TLOB wF1
MambaLOB (base)	70k	0.595	0.508	89%
+ bidirectional	135k	0.597	0.509	90%
+ Conv (ConvMambaLOB)	144k	0.602	0.518	90%
+ both	209k	0.587	0.508	88%
TLOB (SOTA)	1.80M	0.666	0.570	100%

The result is an efficiency trade-off rather than an accuracy win: TLOB still leads weighted *and* macro F_1 in every (instrument, horizon) cell; our variants win none, trailing by ≈ 0.06 – 0.08 wF1. The contribution is the trade-off: **ConvMambaLOB recovers $\sim 90\%$ of the transformer’s quality with $12.5\times$ fewer parameters.** Among our own designs the hybrid *spatial-convolution* front-end is the decisive choice (best or near-best in all four cells); bidirectionality is roughly neutral on average (it helps BANKNIFTY $k=100$ by $+0.037$ but hurts NIFTY $k=10$), and *stacking* both is the weakest Mamba variant — more parameters, no gain, consistent with mild overfitting on the single-seed, one-month NSE sample. Multi-seed confidence intervals (Tier C, below) test which of these effects survive.

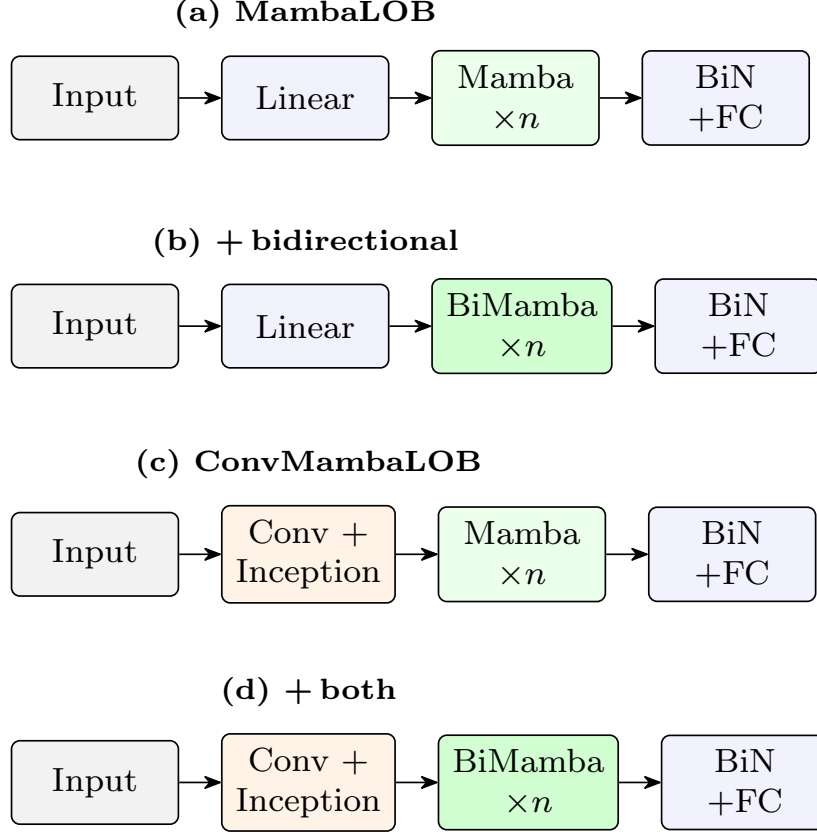


Figure 12: The four Tier-B variants. (a) base MambaLOB; (b) bidirectional Mamba (forward + time-reversed scan, averaged); (c) ConvMambaLOB (spatial conv front-end $\rightarrow$ Mamba core); (d) both combined. All share the Bilinear-Normalization head. Variant (c) was the best of the four; (d) the weakest (Table 9).

3.6.5 Statistical Significance and Calibration (Tier-C)

We re-ran the headline configurations over **3 seeds** (48 runs) and tested each claim two ways: a *paired* test across the 12 (seed, instrument, horizon) cells, and a *per-sample* paired bootstrap + McNemar test on a representative cell (NIFTY, $k=100$). Table 10 summarises.

Table 10: Tier-C significance (weighted- F_1). Across-seed: paired t -test over 12 cells. Per-sample: paired bootstrap 95% CI + McNemar at NIFTY $k=100$.

Claim	Across-seed Δ (p)	Per-sample Δ	Verdict
Features help (Mamba a11 vs base40)	+0.084 ($p=0.001$)	+0.043 (CI excl. 0)	significant
ConvMambaLOB vs base MambaLOB	+0.005 ($p=0.25$)	+0.015 (CI excl. 0)	not robust
Mamba / ConvMamba vs TLOB	-0.07 ($p<0.001$)	-0.087 (CI excl. 0)	TLOB ahead (sig.)

The microstructure feature engineering is significant by every test. The effect is large (an across-seed gain of +0.084 weighted- F_1 , winning 11 of 12 cells) and is especially pronounced on BANKNIFTY, where the raw **base40** input collapses to roughly 0.45 while the engineered **a11** set reaches about 0.61. This is the strongest data-science result of the study: the engineered features are what make the NSE signal learnable.

The ConvMambaLOB gain, by contrast, is real in direction but not statistically robust. It wins 8 of 12 cells and is positive in the per-sample test, yet across seeds the +0.005 average lies within seed-to-seed noise ($p = 0.25$). The per-sample test flags it as significant only because the large sample ($n = 341,214$) renders any tiny difference significant; we therefore report it as a small, consistent-in-direction but not-significant improvement. TLOB’s accuracy lead, on the other hand, is statistically robust ($p < 0.001$, winning all 12 cells), which confirms that the contribution of the proposed models is the efficiency

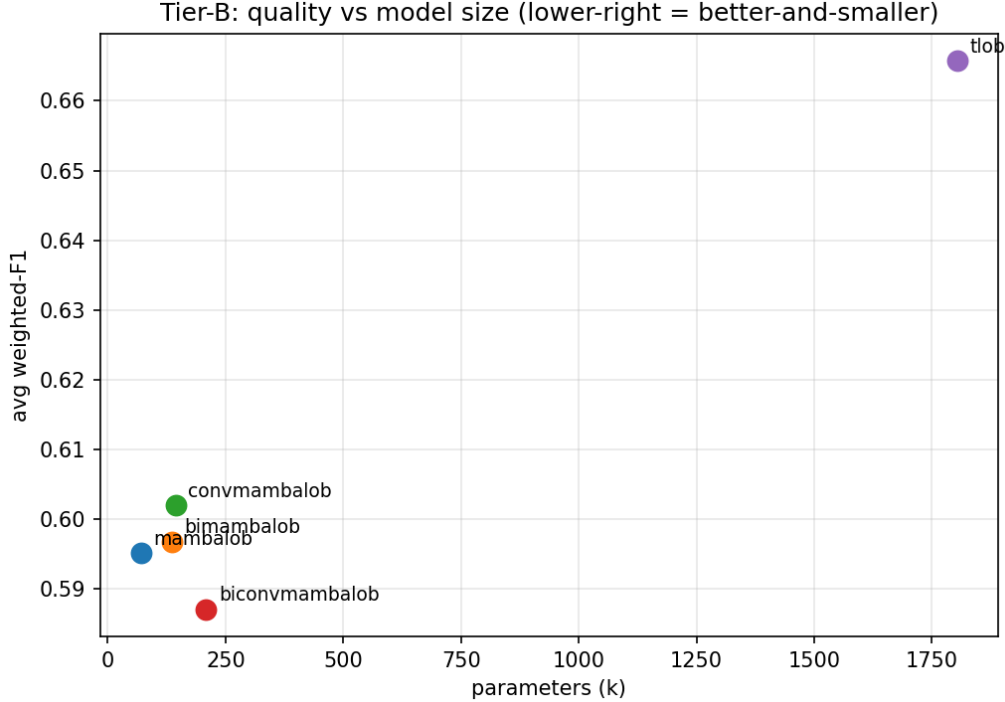


Figure 13: Quality vs. model size for the Tier-B architectures (average weighted- F_1 against parameter count). The Mamba family clusters at the low-parameter end near TLOB’s accuracy; ConvMambaLOB is the best of the proposed variants. The lower-right region (better-and-smaller) is the design target.

trade-off (competitive accuracy at $12\text{--}25\times$ fewer parameters) rather than parity. On calibration, the proposed model is already well-calibrated (test ECE 0.041; the NLL-optimal temperature is $T = 1.00$, so no scaling is needed), so its predicted probabilities can be used directly in the cost-aware backtest. A methodological point worth recording is that per-sample tests over-detect at large n , so the across-seed paired test is the more meaningful measure of a generalizable improvement.

3.6.6 Hyperparameter Optimization and Class Imbalance (Tier-C)

To rule out that the model comparisons merely reflect lucky or unlucky default settings, we ran an Optuna (TPE) hyperparameter search for the two proposed models on NIFTY $k=100$, tuning learning rate, batch size, weight decay, and the Mamba width/depth (≈ 25 trials each). Two results stand out. First, tuning gives only a *modest* improvement (validation weighted- F_1 rises by $\approx 0.012\text{--}0.015$), and the tuned learning rates ($\approx 2.4\text{--}2.9 \times 10^{-4}$) sit close to the 3×10^{-4} default, confirming the prior settings were already near-optimal. Second, and more telling, the tuned *base* MambaLOB (val 0.675) slightly *beats* the tuned ConvMambaLOB (val 0.669), and its best configuration is *smaller* than the default (one Mamba layer, $d_{\text{model}}=32$). Under an equal tuning budget the simpler, tinier model wins — the same conclusion as the multi-seed test, and the strongest form of the efficiency argument: a minimal selective-SSM is the sweet spot, not the convolutional hybrid. (TLOB was excluded from the search for compute reasons, so this is a within-family comparison.)

We also studied four remedies for the class imbalance against the default class-weighted cross-entropy (Table 11), using ConvMambaLOB on NIFTY. At the shorter, more balanced horizon ($k=10$) no remedy beats the default. At $k=100$, where the Stationary class is only $\approx 5\%$ of samples, focal loss gives a small gain (macro- F_1 $0.475 \rightarrow 0.481$, weighted $0.627 \rightarrow 0.646$), but it does so by modelling the dominant directional classes better while almost ignoring Stationary (recall 0.11). Strategies that explicitly raise minority recall — balanced sampling and label smoothing — do lift Stationary recall (to 0.40 and 0.63) but at a clear net cost to overall macro- F_1 . The minority class cannot be recovered without sacrificing accuracy elsewhere; class-weighted cross-entropy remains a sound default, with focal a marginal alternative at long horizons.

Table 11: Class-imbalance strategies (ConvMambaLOB, NIFTY): macro- F_1 and per-class recall.

Strategy	$k=10$			$k=100$		
	mF1	rec. Stat	wF1	mF1	rec. Stat	wF1
weighted CE (default)	0.525	0.389	0.544	0.475	0.217	0.627
focal	0.525	0.447	0.542	0.481	0.108	0.646
label smoothing	0.523	0.412	0.542	0.397	0.632	0.510
balanced sampling	0.520	0.396	0.539	0.443	0.396	0.578
plain CE	0.499	0.209	0.531	0.442	0.000	0.626

3.7 Originality and Innovation in Approach

3.7.1 Creative Framing of the Problem

The first ML-ready NSE index-futures LOB benchmark, framed honestly for snapshot depth data (an “event” is a change in the top-of-book), and a transaction-cost-relative labelling scheme that exposes when signal is economically meaningful.

3.7.2 Experimental Innovations

MambaLOB (selective SSM for LOB classification); **ConvMambaLOB**, a hybrid that fuses DeepLOB’s spatial convolutions with a linear-time selective-SSM temporal core (Section 3.6); a controlled long-context experiment exploiting Mamba’s linear scaling; and two labelling schemes (fixed vs spread-relative) reported side by side.

3.8 Risk Assessment and Project Management

3.8.1 Progress Tracker vs. Original Timeline

Completed: literature survey; the engineered NSE dataset + pipeline; FI-2010 reproduction (DeepLOB, MLPLOB, TLOB); MambaLOB on FI-2010 + efficiency analysis; the *complete* NSE Scheme-A matrix (all four models, both instruments, four horizons); and the full Phase-2 data-science layer — the microstructure feature ablation (Tier-A), the improved architectures (Tier-B), and the Tier-C rigour study (multi-seed significance, calibration, hyperparameter optimisation, and the class-imbalance comparison). Deferred (next phase): Scheme-B confirmatory training, the cost-aware backtest, and a walk-forward robustness check.

3.8.2 Bottlenecks and Delays

Free-tier GPU throttling (mitigated by moving from Colab to Kaggle, which has a separate quota); a Kaggle P100 incompatible with the latest PyTorch CUDA build (use T4); `mamba-ssm` failing to build under pip build isolation; and session time-outs on long matrices.

3.8.3 Risk Mitigation Strategies

`mamba-ssm` installed with `--no-build-isolation`; per-run S3 upload + pull-on-start makes every long matrix resumable (a time-out costs at most the run in flight); host-agnostic secrets/data so the same notebooks run on Colab or Kaggle.

3.9 Experimental Reproducibility

3.9.1 Data Splits and Control Measures

FI-2010: standard CF_7 (7 train / 1 val / 3 test days). NSE: chronological front-month split (train 12 May–4 June; test 5–12 June). Normalization statistics are computed on training data only; windows and labels never cross a day/contract boundary.

3.9.2 Random Seed Control

A `set_seed` utility seeds Python/NumPy/PyTorch; the headline configurations were run over 3 seeds with across-seed 95% confidence intervals and paired significance tests (Section 3.6, Table 10). All results, metrics JSON, and checkpoints are versioned on S3 under `experiments/`.

3.10 Scalability and Efficiency Considerations

3.10.1 Memory, Time, and Resource Profiling

Table 12: Efficiency at $L=100$, batch 1 (inference).

Model	Params	Latency (ms)	Peak mem (MB)
DeepLOB	191k	1.5	30
MLPLOB	529k	0.2	22
TLOB	1.80M	6.1	27
MambaLOB	68k	1.5	20

Table 13: Scaling with sequence length (batch 32): TLOB $O(L^2)$ vs MambaLOB $O(L)$.

L	TLOB params	TLOB mem / latency	Mamba params	Mamba mem / latency
100	1.8M	49 MB / 11 ms	68k	32 MB / 1.5 ms
400	13.5M	164 MB / 54 ms	68k	69 MB / 1.8 ms
800	51M	400 MB / 164 ms	68k	118 MB / 5.3 ms

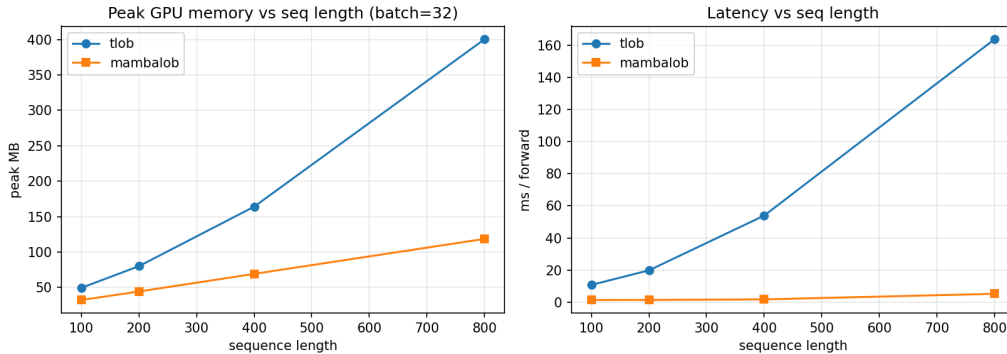


Figure 14: Sequence-length scaling: TLOB’s $O(L^2)$ attention versus MambaLOB’s $O(L)$ selective scan. The gap widens sharply with L , the basis of the architecture’s efficiency claim.

MambaLOB’s parameter count is constant in L and its memory/latency grow *linearly*, while TLOB grows *quadratically*. At $L=800$ MambaLOB is $\approx 31\times$ faster, uses $\approx 3.4\times$ less memory, and has $\approx 750\times$ fewer parameters — a clean demonstration of the linear-scaling advantage that motivates the architecture. (FLOP counts for Mamba are under-reported by the profiler because the fused CUDA kernel is not traced; we therefore rely on measured latency and memory.)

3.10.2 Scalability Constraints

Training throughput is bounded by data loading and the sequential scan rather than raw FLOPs; the spread-relative task is degenerate at short horizons; and real-time deployment is out of scope.

3.11 Ethical Considerations and Responsible AI Use

3.11.1 Generative AI Usage Disclosure

Generative-AI assistance (a coding/writing assistant) was used for code scaffolding, debugging, drafting documentation, and report formatting. All generated content and code were reviewed, tested, and validated before inclusion.

3.11.2 Bias and Fairness in Dataset

The data are public benchmark data and self-collected public market data; there is no personal or demographic data. The main “bias” risk is class imbalance, which we address with class-weighted loss and by reporting macro F_1 and baselines.

3.12 Feedback Incorporation and Continuous Improvement

3.12.1 Peer and Mentor Feedback Summary

Feedback drove: separating weighted vs macro F_1 reporting; checking the spread-relative label balance before training (which revealed degeneracy and saved a large, pointless run); and prioritising the data-engineering narrative.

3.12.2 Reflection and Learning Journey

The project built skills across data engineering (binary protocol decoding, cloud collection, robust pre-processing), reproducible deep-learning experimentation, and honest empirical evaluation under cost and compute constraints.

3.13 Chapter Summary and Next Steps

We engineered an NSE index-futures LOB dataset, reproduced three baselines on FI-2010, introduced MambaLOB with a strong efficiency result, and completed the NSE Scheme-A transfer study (where MambaLOB transfers competitively, beating the CNN baseline on NIFTY).

Next steps — Phase 2 (planned, pending mentor discussion). The work so far is strong on data engineering, reproduction, and transfer; Phase 2 deepens the *Data-Science* contribution so the novelty is unambiguous. It has three tiers:

(A) *Microstructure feature engineering (DE→DS bridge; novel)* — **done**, see Section 3.3. Engineered OFI, depth imbalance, micro-price, relative spread, and the Dhan-unique order-count channel, ablated against the raw window. Result: microstructure features help (best set lifts MambaLOB weighted- F_1 by up to +0.068 at $k=100$), order counts help only in combination, and book depth beyond level 10 is uninformative. Multi-seed CIs (Tier C) remain to confirm significance.

(B) *Improved MambaLOB architecture (the headline novelty)* — **done**, see Section 3.6. Built and evaluated (i) **bidirectional Mamba** and (ii) **ConvMambaLOB** (DeepLOB convolutions → linear-time Mamba core), plus their combination, against TLOB. Result: ConvMambaLOB recovers ~90% of TLOB’s weighted- F_1 at $12.5\times$ fewer parameters — an efficiency win, not an accuracy win (TLOB still leads). The spatial-convolution front-end is the decisive design; bidirectionality and stacking give diminishing returns. Multi-seed CIs (Tier C) remain to confirm significance.

(C) *Rigour layer (expected practice)* — **done**, see Section 3.6, Tables 10 and 11. The 3-seed study with paired across-seed and per-sample tests confirmed the feature-engineering gain ($p=0.001$) and the (significant) gap to TLOB, showed the ConvMambaLOB gain is not robust across seeds, and found the proposed model already well-calibrated (ECE 0.041, $T=1.00$). Hyperparameter optimization (Optuna) gave only a modest gain and showed the tuned base MambaLOB — at one layer and $d_{\text{model}}=32$ — matches the tuned hybrid, reinforcing the efficiency story; the class-imbalance study found class-weighted cross-entropy a sound default (focal a marginal alternative at long horizons).

Also carried over from the interim plan: the Scheme-B confirmatory training, the cost-aware backtest, a walk-forward robustness check, and consolidation into the final report. A standalone `docs/phase2.plan.md` tracks this in detail.

Appendices (planned). Detailed per-(model, symbol, horizon) result tables; confusion-matrix and scaling figures; the notebook→result map; and reproducibility artefacts (**requirements**, seeds, S3 paths).

References

- [1] Z. Zhang, S. Zohren, and S. Roberts. “DeepLOB: Deep Convolutional Neural Networks for Limit Order Books.” *IEEE Transactions on Signal Processing*, 67(11):3001–3012, 2019.
- [2] L. Berti and G. Kasneci. “TLOB: A Novel Transformer Model with Dual Attention for Price Trend Prediction with Limit Order Book Data.” *arXiv preprint* arXiv:2502.15757, 2025.
- [3] A. Ntakaris, M. Magris, J. Kannianen, M. Gabbouj, and A. Iosifidis. “Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods.” *Journal of Forecasting*, 37(8):852–866, 2018.
- [4] M. Prata et al. “LOB-Based Deep Learning Models for Stock Price Trend Prediction: A Benchmark Study.” *Artificial Intelligence Review*, 57:116, 2024 (arXiv:2308.01915, 2023).
- [5] A. Gu and T. Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces.” *arXiv preprint* arXiv:2312.00752, 2023.
- [6] A. Gu, K. Goel, and C. Ré. “Efficiently Modeling Long Sequences with Structured State Spaces.” *International Conference on Learning Representations (ICLR)*, 2022.
- [7] R. Cont, A. Kukanov, and S. Stoikov. “The Price Impact of Order Book Events.” *Journal of Financial Econometrics*, 12(1):47–88, 2014.
- [8] S. Stoikov. “The micro-price: a high-frequency estimator of future prices.” *Quantitative Finance*, 18(12):1959–1966, 2018.

A Detailed Result Tables

This appendix lists the complete per-run results behind the summary tables in Chapter 3. All NSE runs use Scheme A (fixed $\alpha=10^{-5}$) unless noted; metrics are on the held-out test split. Checkpoints and metrics JSON for every run are versioned on S3 under `experiments/<area>/`.

A.1 NSE Scheme-A transfer matrix (32 runs)

Table 14: NSE Scheme-A: all four models $\times$ two instruments $\times$ four horizons. Lift = wF1 minus the best naive baseline.

Model	Symbol	k	Params	wF1	mF1	Acc	MCC	Lift
deeplob	BANKNIFTY	10	191k	0.4574	0.4570	0.4578	0.1896	0.1165
deeplob	BANKNIFTY	20	191k	0.4625	0.4308	0.4553	0.1682	0.0811
deeplob	BANKNIFTY	50	191k	0.4936	0.3884	0.4697	0.1353	0.0543
deeplob	BANKNIFTY	100	191k	0.4962	0.3680	0.4796	0.1334	0.0332
mambalob	BANKNIFTY	10	68k	0.4514	0.4483	0.4511	0.1765	0.1105
mambalob	BANKNIFTY	20	68k	0.4687	0.4163	0.4638	0.1496	0.0873
mambalob	BANKNIFTY	50	68k	0.4659	0.3676	0.4292	0.1271	0.0266
mambalob	BANKNIFTY	100	68k	0.4784	0.3536	0.4461	0.0865	0.0154
mlplob	BANKNIFTY	10	529k	0.3544	0.3562	0.3579	0.0668	0.0135
mlplob	BANKNIFTY	20	529k	0.4071	0.3650	0.4158	0.1092	0.0257
mlplob	BANKNIFTY	50	529k	0.4638	0.3713	0.4261	0.1997	0.0245
mlplob	BANKNIFTY	100	529k	0.4698	0.3514	0.3908	0.2110	0.0068
tlob	BANKNIFTY	10	1.80M	0.6393	0.6323	0.6354	0.4568	0.2984
tlob	BANKNIFTY	20	1.80M	0.6498	0.5983	0.6355	0.4433	0.2684
tlob	BANKNIFTY	50	1.80M	0.6890	0.5535	0.6489	0.4523	0.2497
tlob	BANKNIFTY	100	1.80M	0.7171	0.5255	0.7044	0.4696	0.2541
deeplob	NIFTY	10	191k	0.4440	0.4232	0.4524	0.1512	0.0947
deeplob	NIFTY	20	191k	0.4814	0.4252	0.4896	0.1690	0.1042
deeplob	NIFTY	50	191k	0.4933	0.3949	0.4897	0.1386	0.0681
deeplob	NIFTY	100	191k	0.5162	0.3885	0.5037	0.1428	0.0655
mambalob	NIFTY	10	68k	0.5217	0.4963	0.5236	0.2684	0.1724
mambalob	NIFTY	20	68k	0.5514	0.4919	0.5460	0.2908	0.1742
mambalob	NIFTY	50	68k	0.5750	0.4643	0.5316	0.2895	0.1498

continued on next page

Table 14 (continued)

Model	Symbol	k	Params	wF1	mF1	Acc	MCC	Lift
mambalob	NIFTY	100	68k	0.5940	0.4433	0.5845	0.2723	0.1433
mlplob	NIFTY	10	529k	0.4639	0.4397	0.4657	0.1798	0.1146
mlplob	NIFTY	20	529k	0.5324	0.4430	0.5654	0.2750	0.1552
mlplob	NIFTY	50	529k	0.6042	0.4698	0.6141	0.3322	0.1790
mlplob	NIFTY	100	529k	0.6398	0.4588	0.6591	0.3834	0.1891
tlob	NIFTY	10	1.80M	0.5989	0.5789	0.5901	0.3877	0.2496
tlob	NIFTY	20	1.80M	0.6266	0.5702	0.6091	0.4045	0.2494
tlob	NIFTY	50	1.80M	0.6870	0.5620	0.6641	0.4557	0.2618
tlob	NIFTY	100	1.80M	0.7093	0.5316	0.6989	0.4673	0.2586

A.2 Tier-A microstructure feature ablation (12 runs)

Table 15: Tier-A feature ablation (MambaLOB and a TLOB spot-check, NIFTY).

Model	Symbol	Feat. set	k	F	Params	wF1	mF1	Acc
mambalob	NIFTY	all	10	66	70k	0.5418	0.5209	0.5420
mambalob	NIFTY	base40	10	40	68k	0.5175	0.4957	0.5130
tlob	NIFTY	base40	10	40	1.80M	0.5989	0.5789	0.5901
mambalob	NIFTY	depth80	10	80	71k	0.5149	0.4884	0.5198
mambalob	NIFTY	micro	10	46	69k	0.5455	0.5217	0.5513
tlob	NIFTY	micro	10	46	1.80M	0.6063	0.5852	0.6003
mambalob	NIFTY	orders60	10	60	70k	0.4488	0.4259	0.4567
mambalob	NIFTY	all	100	66	70k	0.6368	0.4757	0.6285
mambalob	NIFTY	base40	100	40	68k	0.5685	0.4214	0.5606
mambalob	NIFTY	depth80	100	80	71k	0.5706	0.4218	0.5698
mambalob	NIFTY	micro	100	46	69k	0.6080	0.4561	0.5984
mambalob	NIFTY	orders60	100	60	70k	0.5612	0.4208	0.5458

A.3 Tier-B architecture comparison (20 runs)

Table 16: Tier-B: improved MambaLOB variants vs. TLOB on the **all** feature set.

Model	Symbol	k	Params	wF1	mF1	Acc	MCC	Train (s)
biconvmambalob	BANKNIFTY	10	209k	0.6136	0.6094	0.6112	0.4223	2575
bimambalob	BANKNIFTY	10	135k	0.6097	0.6051	0.6074	0.4142	1372
convmambalob	BANKNIFTY	10	144k	0.6192	0.6139	0.6174	0.4256	2235
mambalob	BANKNIFTY	10	70k	0.6064	0.6011	0.6044	0.4084	656
tlob	BANKNIFTY	10	1.80M	0.6319	0.6264	0.6262	0.4561	3715
biconvmambalob	BANKNIFTY	100	209k	0.5701	0.4252	0.5274	0.2947	1706
bimambalob	BANKNIFTY	100	135k	0.6258	0.4630	0.5999	0.3204	839
convmambalob	BANKNIFTY	100	144k	0.6113	0.4560	0.5517	0.3222	1471
mambalob	BANKNIFTY	100	70k	0.5889	0.4364	0.5667	0.2699	523
tlob	BANKNIFTY	100	1.80M	0.7121	0.5315	0.6812	0.4590	3079
biconvmambalob	NIFTY	10	209k	0.5404	0.5212	0.5400	0.2978	3022
bimambalob	NIFTY	10	135k	0.5286	0.5033	0.5376	0.2850	1523
convmambalob	NIFTY	10	144k	0.5471	0.5273	0.5472	0.3071	2246
mambalob	NIFTY	10	70k	0.5423	0.5209	0.5439	0.2995	738
tlob	NIFTY	10	1.80M	0.6052	0.5845	0.5988	0.3958	5007
biconvmambalob	NIFTY	100	209k	0.6243	0.4767	0.5821	0.3341	1728
bimambalob	NIFTY	100	135k	0.6230	0.4644	0.6238	0.3323	705
convmambalob	NIFTY	100	144k	0.6303	0.4762	0.6116	0.3416	1448
mambalob	NIFTY	100	70k	0.6430	0.4728	0.6474	0.3527	294
tlob	NIFTY	100	1.80M	0.7138	0.5375	0.7037	0.4764	3113

A.4 Tier-C multi-seed study (48 runs)

Table 17: Tier-C: headline configurations over 3 seeds (significance/CIs in Table 10).

Model/feat.	Symbol	k	Seed	wF1	mF1	Acc	MCC
convmambalob/all	BANKNIFTY	10	0	0.6169	0.6111	0.6157	0.4209
convmambalob/all	BANKNIFTY	10	1	0.6103	0.6067	0.6072	0.4221
convmambalob/all	BANKNIFTY	10	2	0.6102	0.6065	0.6074	0.4230
mambalob/all	BANKNIFTY	10	0	0.6068	0.6016	0.6046	0.4079
mambalob/all	BANKNIFTY	10	1	0.6121	0.6077	0.6096	0.4180
mambalob/all	BANKNIFTY	10	2	0.6082	0.6037	0.6062	0.4119
mambalob/base40	BANKNIFTY	10	0	0.4511	0.4488	0.4508	0.1807
mambalob/base40	BANKNIFTY	10	1	0.4399	0.4379	0.4429	0.1684
mambalob/base40	BANKNIFTY	10	2	0.4483	0.4434	0.4522	0.1737
tlob/all	BANKNIFTY	10	0	0.6319	0.6264	0.6262	0.4561
tlob/all	BANKNIFTY	10	1	0.6343	0.6289	0.6289	0.4587
tlob/all	BANKNIFTY	10	2	0.6422	0.6353	0.6386	0.4610
convmambalob/all	BANKNIFTY	100	0	0.5876	0.4392	0.5092	0.3066
convmambalob/all	BANKNIFTY	100	1	0.6244	0.4611	0.5835	0.3222
convmambalob/all	BANKNIFTY	100	2	0.6053	0.4524	0.5314	0.3247
mambalob/all	BANKNIFTY	100	0	0.6126	0.4509	0.5903	0.2920
mambalob/all	BANKNIFTY	100	1	0.5952	0.4374	0.5870	0.2872
mambalob/all	BANKNIFTY	100	2	0.6096	0.4498	0.5922	0.2882
mambalob/base40	BANKNIFTY	100	0	0.4671	0.3469	0.4270	0.0869
mambalob/base40	BANKNIFTY	100	1	0.4598	0.3421	0.4182	0.0792
mambalob/base40	BANKNIFTY	100	2	0.4957	0.3627	0.4763	0.0921
tlob/all	BANKNIFTY	100	0	0.7121	0.5315	0.6812	0.4590
tlob/all	BANKNIFTY	100	1	0.7211	0.5260	0.7180	0.4795
tlob/all	BANKNIFTY	100	2	0.7096	0.5279	0.6781	0.4566
convmambalob/all	NIFTY	10	0	0.5440	0.5254	0.5422	0.3032
convmambalob/all	NIFTY	10	1	0.5404	0.5219	0.5390	0.2980
convmambalob/all	NIFTY	10	2	0.5468	0.5264	0.5470	0.3060
mambalob/all	NIFTY	10	0	0.5413	0.5204	0.5421	0.2977
mambalob/all	NIFTY	10	1	0.5334	0.5092	0.5395	0.2882
mambalob/all	NIFTY	10	2	0.5339	0.5083	0.5435	0.2927
mambalob/base40	NIFTY	10	0	0.5339	0.5114	0.5278	0.2858
mambalob/base40	NIFTY	10	1	0.5265	0.5008	0.5290	0.2766
mambalob/base40	NIFTY	10	2	0.5421	0.5246	0.5321	0.3068
tlob/all	NIFTY	10	0	0.6052	0.5845	0.5988	0.3958
tlob/all	NIFTY	10	1	0.6069	0.5837	0.6046	0.3974
tlob/all	NIFTY	10	2	0.6067	0.5845	0.6032	0.3967
convmambalob/all	NIFTY	100	0	0.6425	0.4829	0.6269	0.3525
convmambalob/all	NIFTY	100	1	0.6250	0.4758	0.5966	0.3273
convmambalob/all	NIFTY	100	2	0.6167	0.4645	0.5931	0.3163
mambalob/all	NIFTY	100	0	0.6271	0.4731	0.6121	0.3332
mambalob/all	NIFTY	100	1	0.6179	0.4482	0.6305	0.3172
mambalob/all	NIFTY	100	2	0.6175	0.4602	0.6066	0.3107
mambalob/base40	NIFTY	100	0	0.5845	0.4343	0.5753	0.2684
mambalob/base40	NIFTY	100	1	0.5724	0.4284	0.5581	0.2638
mambalob/base40	NIFTY	100	2	0.5851	0.4387	0.5641	0.2640
tlob/all	NIFTY	100	0	0.7138	0.5375	0.7037	0.4764
tlob/all	NIFTY	100	1	0.7189	0.5485	0.6991	0.4820
tlob/all	NIFTY	100	2	0.6982	0.5334	0.6701	0.4556